consiness
success connects

SAP S/4HANA · ON PREMISE · CLASSIC ABAP

Documizer

Documentation, transfer and AI analysis for SAP development and customizing

A single ABAP report, installed in seconds. It documents what your system **really** contains — and, if you want, carries it into another system. No transport, no abapGit, no additional repository objects.

3
reports: customizing, development, check configuration

1
source per report — copy and paste into SE38

20+
object types that can be transferred

0
transports, installations or extra objects

Product documentation · September 2026
Christian Gathmann · gathmann@consiness.de
consiness GmbH & Co. KG

At a glance

Anyone responsible for an SAP system knows the problem: the documentation describes what was once planned. What is actually in the system is known only to the system itself. The Documizer asks it — and turns the answer into a document that people can read, systems can read back in, and AI assistants can analyse.

`ZDOCUMIZER_IMG`

Customizing

Reads a structure tree of the Implementation Guide, picks up SAP's documentation for each IMG activity and follows the customizing behind it down to the table contents. Creates a BC set on request.

`ZDOCUMIZER_DEV`

Development

Starts at a package or a single object — report, class, table, CDS, RAP, IDoc, UI5 app. Delivers fields, methods, texts, source code, where-used lists. And imports.

`ZDOCUMIZER_ATC`

Check configuration

Documents ATC and Code Inspector variants: which checks run, at which priority, customer prioritisation, pseudo comment, pragma and exemptions.

What sets the Documizer apart

- **Installed in seconds.** Each report is a single source. Paste it into SE38, activate, done — even where nothing may be installed and no transport may be imported.
- **One format for everything.** The XML is complete, versioned and can be read back without loss. The HTML is self-contained and will still open years from now without any rework.
- **Documentation that can go back into a system.** From the same document, the development report creates tables, classes, CDS views, RAP objects, IDoc types and UI5 apps in another system.
- **Never silently incomplete.** If a limit takes effect, the document says so. Every run can check itself: serialise, read back, compare.
- **Safe in a customer system.** SAP's authorization checks stay switched on. Customizing is never written directly; it is offered as a BC set for review.

Contents

At a glance 2
1 Why Documizer 4
2 Six scenarios 5
3 Installed in seconds 8
4 Documenting customizing: ZDOCUMIZER_IMG 9
5 Transferring customizing: BC sets and the list of gaps 11
6 Documenting development: ZDOCUMIZER_DEV 12
7 Documenting check standards: ZDOCUMIZER_ATC 14
8 Output: XML and HTML 15
9 Import: bringing development into another system 16
10 Security and operation 18
11 Limits and outlook 19
12 Contact 20

1 Why Documizer

SAP systems grow over years. Packages, classes, CDS views, customizing entries — and with every release the documentation drifts further away from reality.

Four problems from everyday project work

Problem	What it costs
Documentation is out of date. It describes the design, not the state of the system.	Onboarding takes weeks; faults are looked for in the wrong place.
Customer systems are sealed off. No installation, no transports, abapGit is not an option.	Tools that are a matter of course in your own system are simply not available at the customer.
Knowledge sits inside the system. Who uses what, which fields are read, which checks apply — that lives in tables, not in documents.	Impact analyses are estimated instead of looked up.
AI needs context. An assistant is only as helpful as the information it is given.	Individual code snippets are rarely enough for research and troubleshooting.

The answer

The Documizer reads straight from the running system — through SAP's standard APIs wherever they exist. It writes nothing in the process, unless you explicitly ask it to import. The result is a document structured into chapters, in two formats:

- **XML** — complete, versioned, machine-readable. The transfer format for the import and the ideal input for AI assistants.
- **HTML** — a single file with a table of contents, embedded styling and no JavaScript. For reading, passing on, filing away.

> **The principle**
> One source per report. One format for people, systems and AI. No installation in the target system.

2 Six scenarios

The same report, the same file — six tasks that would otherwise each call for a tool of their own.

Documenting development

THE TASK

A package, an application or a single object is to be described completely — for handover, maintenance, audit, or onboarding new colleagues.

WHAT THE DOCUMIZER DELIVERS

- Short texts, author and package for each object, and the object documentation on request
- Field lists of tables, structures, views and CDS views with data elements and keys
- Classes with superclass, interfaces, components and local classes — including the handlers of a RAP behavior pool
- Source code, where-used list with the exact occurrences, and the objects that are used

> **Your benefit**
> Documentation that matches the system — regenerated at the push of a button instead of being maintained by hand.

Documenting IMG settings

THE TASK

The customizing of an area or a project is to be recorded so that it can be followed — with explanation and actual values.

WHAT THE DOCUMIZER DELIVERS

- Structure tree of the Implementation Guide with all IMG activities
- SAP’s standard documentation for each activity, rendered as readable text
- The customizing tables behind it with field list and content — customer entries only, if you wish
- Project status and memo from an IMG project

> **Your benefit**
> The customizing documentation comes out of the system itself. SAP’s explanation sits right next to your values.

Transferring development

THE TASK

Objects are to go into another system — from a sandbox into the customer system, say — where neither transports can be imported nor abapGit installed.

WHAT THE DOCUMIZER DELIVERS

- Definition data in the XML: the same structures SAP returns when reading and expects when creating
- Import of more than 20 object types, from the domain to the UI5 application
- Test run as the default, existing objects are left untouched
- Package rules old → new; request and task are assigned automatically

> **Your benefit**
> Development travels as a single file. The target system only needs the Documizer itself.

Transferring customizing via BC set

THE TASK

Customizing from one system is to be reproduced in another — in a controlled and traceable way.

WHAT THE DOCUMIZER DELIVERS

- The IMG document becomes a Business Configuration Set
- Activation happens in SCPR20: with a log, a transport request, a preview and a way back
- A list of gaps names the tables still missing in the target system — the development report exports exactly those

> **Your benefit**
> No report writes into your customizing tables. You decide what is activated.

AI-assisted analysis

THE TASK

An AI assistant such as Claude is to help with research, troubleshooting or documentation — and needs the full context for that.

WHAT THE DOCUMIZER DELIVERS

- A structured XML with source code, field lists, where-used lists and customizing in one piece
- Unambiguous chapters and object references that questions can refer to
- Notes wherever something was shortened — the assistant knows what it cannot see

> **Your benefit**
> Instead of copying individual snippets, you hand the assistant the whole application — and get well-founded answers.

Making check standards transparent

THE TASK

Which ATC checks run, at which priority, and how can findings be suppressed? The answer is spread across clusters and classes.

WHAT THE DOCUMIZER DELIVERS

- Check variants with categories, checks and all findings
- Priority of the check class next to the customer-specific prioritisation
- Pseudo comment and pragma for each finding, exemptions granted

> **Your benefit**
> Code standards become readable and comparable between systems.

3 Installed in seconds

Each report is a single source file. There is nothing to transport and nothing to configure.

1. Call SE38, choose a program name (`ZDOCUMIZER_DEV`, for instance), select **Create**, type *Executable program*.
2. Paste the source, save, activate.
3. Enter the program title under *Goto* → *Attributes* — the only thing set by hand.

Why this works

No global classes, no text elements, no tables of its own, no Simple Transformation, no message class. All screen texts live in the source — in English, and in German for a German logon.

XML is written and read through the kernel classes `cl_sxml_string_writer` and `cl_sxml_string_reader`. They exist in every system.

Report documentation in the system (optional)

The helper report `ZDOCUMIZER_DOCU` writes the SAP report documentation for the three reports — visible in SE38 under *Documentation* and through the info icon on the selection screen. It is not needed for the reports to work; it makes sense when other colleagues work with the Documizer in that system.

System requirements

Item	Requirement
Platform	SAP S/4HANA on premise, classic ABAP; verified on SAP_BASIS 758
Authorization for documentation	Read access to the repository; `S_TABU_NAM` for customizing content
Authorization for import	`S_DEVELOP` for the objects and packages to be created — SAP's checks stay switched on
Operation	Dialog and background; in the background, output goes to the application server

4 Documenting customizing: ZDOCUMIZER_IMG

The IMG report reads a structure tree of the Implementation Guide, assigns its IMG activity to each node and goes all the way down to the customizing entries.

What the document contains

```
1  <root node>                          structure chapter
   1.1 <sub-node>                       structure chapter
       1.1.1 <IMG activity>             activity chapter
             ├─ Description             SAP documentation, made readable
             ├─ Technical assignment    activity, transaction, doc object
             ├─ Project status / memo   in project mode
             ├─ Object per assignment   field list, selection method, row count
             ├─ Customizing content     data according to the volume settings
             ├─ Transport history       requests for the table
             └─ Objects used            program behind the transaction
```

Selection screen

Field	Default	Effect
Structure ID (GUID) / or search term for root		Entry point. The report's own value help shows root nodes with text, node count and change date — instead of 500 bare GUIDs
Project (status/memo) / project nodes only		Overlays the tree with an IMG project, trims it to that project's nodes
Max. depth / max. chapters	0 / 300	Size of the tree; 0 = all
Resolve substructures	on	Decisive: without it the run ends after the first level
Standard documentation / object assignment	on / on	SAP documentation per activity, assigned customizing objects
Read customizing content	on	Field list and table contents
Customer entries only / max. rows	on / 500	Volume control, see below
Transport history / outgoing references	off / off	Requests per table; program behind the transaction

Only what is yours

Customizing tables range from three rows to millions. With *customer entries only*, two methods work side by side: the **namespace** of the first key field (Y, Z, `/`) and the **occurrence in transport requests** for that table. That way entries that were never transported, and keys outside the customer namespace, are found as well.

Which method applied is stated at the object — not only in the log. If the read authorization is missing, the object appears with a note instead of silently empty.

Project mode

An IMG project carries status, progress and a memo for each activity. The report overlays the tree with it and can trim the tree to the nodes being worked on. Structure chapters remain as long as a project node hangs below them; chapter numbers keep their position, and the gaps show what was left out.

Measured

Project mode with 351 chapters: **0.48 seconds**. The entry point is derived from the project data — instead of walking the entire guide.

5 Transferring customizing: BC sets and the list of gaps

The IMG report in reverse: a document becomes a Business Configuration Set in the target system. Nothing is written yet.

Why a BC set and not direct access

The customizing comes into being only when someone activates the set in **SCPR20**. Activation runs with a log, with a transport request, with a preview and with a way back. A report writing straight into customizing tables would have none of that.

Blocked when	Reason
The table is unknown in the target system	A set pointing at a missing table cannot be activated
Delivery class is not C, G or E	Anything else is not a customizing table
Content was shortened during export	Shortened content would look complete in the set — on activation the rest would vanish

The list of gaps closes the loop

Customer-specific customizing tables often do not exist yet in the target system — they are development. The IMG report therefore writes a **list of gaps** naming the objects it skipped. In the source system, the development report takes that list as its entry point and exports exactly those objects. Once they are imported, the BC set is created for them too.

```
target system   IMG report    create BC set  →  gaps: Z_SKIP_IND, ZV_SKIP_IND
source system   DEV report    entry point AL →  export with definition data
target system   DEV report    import         →  table and view created
target system   IMG report    create BC set  →  complete
```

> **Your benefit**
> The target system says for itself what is missing. Only what is genuinely absent there is requested.

6 Documenting development: ZDOCUMIZER_DEV

The entry point is a pair of type and name, as in the Object Navigator. A package yields package → object type → object; a single object yields the object, everything it uses, and everyone that uses it.

Entry points

Code	Entry point	Code	Entry point
K	Package (default)	W	CDS view
P	Program / report	MX	Metadata extension
C	Class or interface	BD	Behavior definition
FF	Function module	SD	Service definition
F	Function group	SB	Service binding
T	Table / structure	IS	IDoc segment type
V	View	IT	IDoc basic type
D	Data element	U5	UI5 application
DO	Domain	AL	Worklist from a list of gaps

Selection screen

Field	Default	Effect
Subpackages recursively / include local packages	on / off	Scope in package mode
Object type / object name		Filter on the object list
Max. objects	2000	The total count is recorded in the document
Object documentation / details	on / on	Documentation; fields, methods, text elements, fixed values …
Output source code / max. lines per object	off / 300	Source code; any shortening is stated in the document header
Where-used list / also for used objects	off / off	Who uses the object — with occurrences and fields
Outgoing references / reference depth	off / 1	What the object uses, and how far that is followed
Usage depth	0	Document the users found as chapters of their own
Exclude SAP standard	off	Only custom development in the reference lists
Definition data for re-import	on	Technical definition for the import

Details per object type

Object type	In the document
Program	Title, selection texts, text symbols, headings, object documentation

Object type	In the document
Class, interface	Superclass, interfaces, components, implemented interface methods, local classes with their methods; source code as one continuous source
Function module, function group	Interface with all parameters; list of modules
Table, structure	Field list with data element or built-in type, and the key
View	Field list, base tables with join conditions
CDS view	Field list of the entity, objects used, source code
Behavior definition, service definition, metadata extension	Description and source code
Service binding	Binding type, linked service definition, publication state
IDoc basic type	Segment hierarchy (WE60), message types (WE82), processing modules (WE57)
IDoc segment	Fields with the data elements that WE31 shows
UI5 application	Description; with definition data, all files including the directory tree
Data element, domain	Field labels; fixed values

References in both directions

Outgoing references show what an object uses. Because SAP's cross-reference indexes each know only part of the picture, the report also reads the source: `CALL FUNCTION`, `SUBMIT`, table accesses, includes, message classes — and even object names in character literals that are needed at runtime. Every hit is checked against the object catalog before it enters the document.

The **where-used list** shows who uses an object: with the program type, for tables with the access type read or write, with occurrences including line numbers — and for structures with the fields touched at each place. For a table, the views and CDS views built on top of it appear as well.

> **Composition is never limited**
> The volume limits apply to usages — never to what an object is made of. The data elements of a table are always collected in full, otherwise it could not be activated in the target system.

Excluding the SAP standard

The yardstick is the software component of the package — the same line SAP itself draws. Unlike a name check on Z and Y, it also catches customer objects in reserved namespaces and SAP objects that begin with Z.

7 Documenting check standards: ZDOCUMIZER_ATC

The third view: not what is in the system, but what it is measured against.

Mode	Answers
Check variant (V)	What **is** checked — one variant or all of them
Catalog (K)	What **could** be checked — all activated checks

```
1       Check variant    owner, transportable, hidden
1.1       Category
1.1.1       Check        findings with code, priority, customer priority,
                         pseudo comment, pragma, exemptions, text
```

The list of checks in a variant is not stored in readable form in the database; the report reads it through the Code Inspector API. A check named by a variant but no longer present after an upgrade still gets a chapter — with the note *cannot be loaded*.

> **Measured**
> Standard variant `DEFAULT`: **17 checks, 1,198 findings**, roundtrip identical.

8 Output: XML and HTML

Both formats come out of the same chapter model in the same run.

XML — complete, versioned, readable back in

```
<documizer version="1.1" type="DEV">
  <meta>
    <system>DV1</system> <client>100</client> <language>E</language>
    <truncated>false</truncated> <node_limit>2000</node_limit> <node_total>44</node_total>
  </meta>
  <chapters>
    <chapter id="1.1.1" level="3" type="OBJECT">
      <title>Z_TABLE - header data</title>
      <object type="TABL" note="field list"> … </object>
      <source> … </source>
      <definition kind="TABL"> … </definition>
    </chapter>
  </chapters>
</documizer>
```

- **Never unnoticed incomplete.** Every shortening — object limit, row limit, source limit — is stated in the header and at the data block.
- **Comparable between systems.** User, timestamp and client appear only in the header, never in the chapters.
- **Roundtrip-checked.** On request the report serialises, reads back in, serialises again and compares character by character.
- **Extensible.** New content goes into new elements; older documents remain readable.

HTML — readable today and in years to come

A single file with a table of contents, numbered chapters, tables and source code blocks; CSS embedded, no JavaScript. SAP documentation is converted from ITF into readable text with subheadings. Long blocks sit collapsed behind a line stating their name and size.

Storage and file name

Destination	Available	Note
Download to the PC	Dialog	Binary in UTF-8 — special characters are preserved
Storage on the application server	Dialog and background	For large runs in the background

The file name is proposed from system, start object, date and time — `documizer_dev_DV1_z_package1_20260922_101500`, for instance — and can be overwritten freely.

9 Import: bringing development into another system

Read an XML produced in system A and create the objects defined in it in system B — with the same report, without a transport and without abapGit.

What is created

Object type	Created
Domain, data element	with fixed values and texts
Table, structure, secondary index	with technical settings and foreign keys; indexes after the table
View, CDS source	with base tables and join conditions; the SQL view arises on activation
Report, include	with text elements and documentation **in all languages**, GUI status
Class, interface	as the source-based form that SE24 and ADT show
Function group	with includes, modules and their interface
Message class	all messages in all maintained languages
Behavior definition, service definition, metadata extension	as source code
Service binding	with binding type and services — deliberately unpublished
IDoc segment type, IDoc basic type	as a real segment, with the whole hierarchy
UI5 application	with all files and the directory tree, through the UI5 repository API
Screen, maintenance dialog, transaction, view cluster	the complete SM30 apparatus, parameter transactions included

Safe from the start

- **Test run is the default.** Every step reports what it would do — writing happens only once you deliberately clear the flag.
- **What exists stays.** An object that is already there is touched only with *overwrite existing objects*.
- **The right order.** Domains before data elements before tables before views before programs — the report sorts this out itself.
- **Activated selectively.** From the user's inactive objects the import takes only its own; work others have started is left alone.
- **Checked after the run.** Active version, syntax check for programs — anything conspicuous is reported.

Packages, request and task

Four rule lines **old** → **new** steer objects from the source package into a target package — `$TMP` → `Z_PACKAGE1`, for instance. The request follows the package of each individual object. If there is no open task for you under the request, the report creates it without a dialog; without that task the correction system would reject the import with a misleading message.

A log that names causes

For each object it says what happened and why — with the real return code and the message from the SAP function module. When an activation fails, the log names the missing object behind it:

```
PROG Z_REPORT_COPY   created                560 lines
TEXT Z_REPORT_COPY   text elements set      D:5 E:8
DOKU Z_REPORT_COPY   documentation set      languages E
DTEL Z_ACTION        DDIF_DTEL_ACTIVATE rc 8 missing or inactive: domain Z_ACTION_D
```

Measured: a RAP application as a single file

Package run with source code and definition data: **34 definitions** — 16 CDS views, 8 classes, 2 behavior definitions, service definition and binding, tables, domain, data element and 2 UI5 applications with **76 files in 12 folders**. Roundtrip identical.

10 Security and operation

Authorizations remain SAP’s business

The IMG report checks `S_TABU_NAM` before every table access; if the authorization is missing, the document says so. During the import, the SAP function modules that are called check `S_DEVELOP` against object and package — and that check stays switched on. The Documizer is therefore not a tool for creating objects that someone would not be allowed to create by hand.

Runtimes

Run	Scope	Duration
IMG, project mode	351 chapters	0.48 s
DEV, package hierarchy	10 packages / 352 objects	0.23 s
DEV, CDS with source code	4 objects	0.08 s

The most expensive parts are the documentation texts per object and the search for usages; both are controlled by a flag. For full runs: volume limits at 0, background job, storage on the application server.

11 Limits and outlook

Whatever the Documizer cannot recreate faithfully, it says so — instead of doing it halfway.

Topic	State
Customizing content	deliberately not written directly — BC sets serve that purpose
Environment of a Fiori app	ICF nodes, MIME objects and gateway registration are documented, not created
Authorization default values	documented, not transferred (SU22/SU24)
IDoc extension types	not supported yet; basic types and segments are complete
Existing screens	are not overwritten — SAP offers no dialog-free way to do it
Missing packages	are not created automatically; that remains a developer’s decision
Further formats	Markdown, and displaying an XML file without system access, are prepared

12 Contact

Would you like to use the Documizer in your system, document an application or transfer development?

Christian Gathmann
gathmann@consiness.de
consiness GmbH & Co. KG

consiness
success connects